

CS 744: Design and Engineering of Computing Systems

Autumn 2026

Lab 3: the xv6 odyssey

Instructions

- This course is on a **no-plagiarism** diet. All parties involved in plagiarism harakiri will be referred to DADAC and penalized to the maximum extent.
 - The Moss Detective Agency has agreed to conduct all investigations.
<https://theory.stanford.edu/~aiken/moss/>
Byomkesh, Sherlock, Phryne, Marple, and Hercule are on standby.
 - Hardcoding the expected output of execution in the source code will yield **negative** marks.
 - Generative AI (ChatGPT, Gemini, etc.) is not you!
Analytical, critical, and creative thinking are learning outcomes of this course, and source code or any content via GenAI tools cannot be part of your submissions.
 - **Note:** Submission guidelines to be strictly followed; otherwise, your submission will not count.
 - Log in with username your **CSE LDAP ID** or **labuser** on the software lab machines for this lab or on your Linux environment/setup.
 - This file is part of the **lab3.tar.gz** archive, which contains multiple directories with programs associated with the exercise questions listed below.
 - Most questions in this lab are tutorial-style and aim to provide an introduction to tools and files related to system and process information and control.
-

xv6 is a working operating system designed to expose the core ideas behind modern OS design in a controlled and understandable way. Developed at MIT, it is a re-implementation of the classic **Unix Version 6**, redesigned for teaching purposes.

Unlike well established operating systems such as **Linux** or **Windows**, xv6 is intentionally **small, readable, and well-structured**, making it ideal for learning core operating system concepts without the complexity of millions of lines of code.

To install xv6 on Ubuntu, refer to details described in [Appendix A](#).

The Holy Grail for xv6

- [xv6-risc book](#)
- [xv6-risc src booklet](#)

The xv6 source tree is organized to mirror OS subsystems:

- **kernel/** → process management, memory, file system, traps
- **user/** → user programs
- **mkfs/** → file system creation tool

Each directory corresponds to a logical OS component, making exploration and modification systematic. **You will mostly interact with **kernel/** and **user/** directory.**

Task 0: booting xv6

In the xv6 directory on your local machine and try the following:

Open a shell inside docker:

Shell

```
> docker run --rm -it -u $(id -u):$(id -g) -v  
<path_to_xv6_dir>:/xv6 cs744/xv6
```

Start xv6:

Shell

```
> make  
> make qemu  
...  
xv6 kernel is booting  
hart 1 starting  
hart 2 starting  
init: starting sh  
$
```

Try a few commands:

Shell

```
$ ls  
$ cat README  
$ echo hello
```

Exit QEMU using: **Ctrl + A, then X**

Task 1: User (space) programs

xv6 is **intentionally simple** by design. Unlike modern operating systems such as Linux or Windows, xv6 **does not provide a compiler or development tools inside the OS**.
e.g., shell based tools, gcc, editor etc.

This means that commands like `gcc hello.c` will not work **inside** xv6

Instead, xv6 follows a **static build model** – all user programs to be executed using xv6 must be **compiled on the host machine** before the xv6 operating system boots. The user programs and the xv6 OS are bundled into files (`xv6.img` and `fs.img`) to emulate state on disk.

How user programs are set up with xv6?

1. Any user-space program you want to run must be written in the `user/` **directory**.
2. These programs are compiled **along with the kernel** during the xv6 build process.
3. After compilation, a **file system image** is created using `mkfs`.
4. This file system image contains:
 - a. All compiled programs
 - b. Any static files (e.g., `.txt` files)
5. QEMU loads this image when xv6 boots, making the programs available in the xv6 shell.

As a result, **new programs cannot be added at runtime**, they must be included **before booting xv6**.

Note that QEMU starts with two image files – **xv6.img** (the xv6 kernel) and **fs.img** (the root file system with all user files).

IMPORTANT

To ensure your programs in the `user/` **directory** are included in compilation, you need to modify the `Makefile`

- To add a program: Locate the `UPROGS` variable in the Makefile and add your user program by appending `$U/_<filename>\`
- To add static files (like `.txt`): Add them directly to the file system image

```
fs.img: mkfs/mkfs README sample.txt $(UPROGS)
mkfs/mkfs fs.img README sample.txt $(UPROGS)
```

Any changes in any of the user files need to be followed by a clean, compile and build, followed by executing xv6.

A. ¡Hola! xv6()

Write a basic user-space program **hello.c** for xv6 that prints **Hello, World!** on the console and then terminates.

Create **hello.c** inside the **user/** directory. Follow the standard structure of an xv6 user program and use the appropriate xv6 headers, such as **kernel/types.h** and **user/user.h**. You may refer to existing programs in the **user/** directory, such as **forphan.c**, to understand how xv6 user programs are structured and how functions such as **printf()** and **exit()** are used.

Add the new program to the **UPROGS** section of the xv6 Makefile using the same naming convention as the existing user programs (for example, **\$U/_echo**). This ensures that the program is compiled and included in the xv6 file-system image.

Sample Output:

```
Shell
> make
> make qemu
...
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh

$ hello
Hello World!
```

B. Borrowing linux commands

Implement the **clear** command as a user-space program in xv6. Create **clear.c** inside the **user/** directory and make it possible to execute the command directly from the xv6 shell:

[\(A magic code for clear screen available for FREE!! Click now!!\)](#)

Sample Output:

```
Shell
$ hello
Hello World!
```

```
$ clear
| blank |
$
```

Also, Implement **head** and **tail** as separate user-space programs in xv6. Create **head.c** and **tail.c** inside the **user/** directory. Both commands should accept a **file name** and the **number of lines** to process, **head** should print the **first N lines** of the specified file, while **tail** should print the **last N lines**.

Hint: Look up implementation in **cat.c** in the **user/** directory.

Sample Output:

```
Shell
> make
> make qemu
...
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh

$ head dummy 5
This is line number 1.
This is line number 2.
This is line number 3.
This is line number 4.
This is line number 5.

$ tail dummy 4
This is line number 27.
This is line number 28.
This is line number 29.
This is line number 30.
```

Use the existing xv6 file-system interfaces, such as **open()**, **read()**, and **close()**, to access and process the file contents. The programs should handle basic errors such as an incorrect number of arguments, an invalid value of **N**, or failure to open the specified file.

Refer to existing xv6 user programs such as **cat.c** and **echo.c** to understand the standard program structure, command-line arguments, and available user-space functions.

C. From Fork to Finish

Write a user-space program named **cmd.c** that creates a **child process** to execute another xv6 user program specified through the command line. The parent process must wait for the child process to finish before terminating.

Create **cmd.c** inside the **user/** directory and use the existing xv6 process-management interfaces provided through **user/user.h**. The implementation must use the **fork()** system call to create the child process and the **exec()** system call to replace the child process with the program specified by the user.

The program to be executed should be provided as a command-line argument and it should support passing arguments to the program being executed.

Sample Output:

```
Shell
$ cmd ls
.          1 1 1024
..         1 1 1024
README    2 2 2441
cat       2 3 36824
echo      2 4 35656
forktest  2 5 17432
.
.

$ cmd echo hello world
hello world

$ cmd head dummy.txt 3
This is line number 1.
This is line number 2.
This is line number 3.
```

Handle basic errors such as missing command-line arguments, failure of **fork()**, or failure of **exec()**.

D. Copying copy

Implement the **cp** (copy) command as a user-space program in xv6. The program should copy the contents of a **source** file into a **destination** file.

The program should accept a file names as a command-line argument:

Shell

```
$ cp source destination
```

The contents of file2 should be identical to the contents of file1.

The implementation should use the existing xv6 file-system interfaces, particularly **open()**, **read()**, **write()**, and **close()**. Open the source file for reading and create/open the destination file for writing.

E. Grep++

Implement a new user-space program **mgrep** in xv6 that searches for a given pattern across multiple files using multiple child processes. You should **reuse the existing pattern-matching functionality provided by the xv6 grep program** rather than implementing a new pattern-matching algorithm.

The program should accept a file names as a command-line argument:

Shell

```
$ mgrep <pattern> <file1> <file2> ... <fileN>
```

Example:

```
$ mgrep xv6 gt1 gt2 gt3
```

For each input file, the parent process must create **exactly one child process** using **fork()**. Each child acts as an independent worker and is responsible only for searching its assigned file. The worker should open the file, use the existing **grep** pattern-matching logic to search for the specified pattern, and print every matching line along with its process ID in the following format:

Shell

```
$ mgrep <pattern> <file1> <file2> ... <fileN>
(Worker PID: <pid1>) <matching line>
(Worker PID: <pid2>) <matching line>
```

The parent process must wait for worker processes to terminate before exiting. The program should also handle basic errors, including invalid command-line arguments, failure of `fork()`, and failure to open an input file.

Sample Output:

Shell

```
$ mgrep xv6 gt1 gt2 gt3
```

```
(Worker PID: 8) ...matching line...
(Worker PID: 8) ...matching line...
(Worker PID: 8) ...matching line...

(Worker PID: 10) ...matching line...
(Worker PID: 10) ...matching line...

(Worker PID: 9) ...matching line...
(Worker PID: 9) ...matching line...
```

Hint: Refer to the existing `grep` implementation in the `user/` directory for its file-reading and pattern-matching logic.

Follow-up: Handling Interleaved Output

When running `mgrep` on multiple files, you may observe that output from different child processes is **interleaved**. Explain why this happens and why calling `wait()` only after creating all the children does not prevent it.

Modify your program to ensure that the output is not interleaved. You may use either:

1. **Sequential approach:** Create one child, `wait()` for it to finish, and then create the next child. This approach is **accepted for the lab**.
2. **Synchronization:** Explore a suitable synchronization mechanism that allows children to execute concurrently while keeping their output separate. **Optional**.

System Calls

Several source files of xv6 are related to implementation of the system call mechanism and the system call interface.

Following is a list, details of which are needed for understanding xv6's implementation of the system call mechanism.

A keyword list of riscv ISA and xv6 functions and variables is listed in Appendix B.

- `user/user.h`
Contains **system call function declarations**. Any new syscall must be declared here so user programs can use/invoke it.
- `kernel/syscall.h`
Defines **system call numbers**. A new system call must be assigned a unique number here.
- `kernel/syscall.c`
Contains the **system call dispatch table**. This maps syscall numbers to kernel handler functions.
- `kernel/sysproc.c`
Most **process-related system calls** like fork, exit, etc., live here. Any new implementation of any process-related system calls can go here.
- `kernel/proc.h`
IMPORTANT - Declaration of **crucial process-related data structures** like PCB and process states is here.
- `kernel/proc.c`
The **heart of process management**. Contains almost everything that defines a process's **lifecycle** and **behavior**. You will get several examples of functions traversing/using the process list.
- `kernel/syscall.c`
Contains functions used to **safely extract syscall parameters passed from user space**.
- `user/usys.pl`
Every system call needs a **user-space wrapper function**. Produces `usys.S`, which contains assembly code that places syscall number in a register and executes `ecall`.
- `kernel/defs.h`
Contains **forward declarations for kernel functions** shared across multiple kernel files.

All or most of these files will have to be used/updated to implement/modify system calls.

Any changes in any of the kernel files need to be followed by a clean, compile and build, followed by executing xv6

Process background

Each process in xv6 is represented by an internal structure known as process control block (`struct proc`).

- `myproc()`: returns the kernel pointer to the process that is currently running on the CPU.
 - `copyout()`: a helper function that safely copies data from kernel memory into userspace memory (directly writing to user pointers is unsafe since kernel memory layout is different from userspace memory layout).
 - `argint()`: parses an integer argument passed from the userspace to a syscall, and stores it in a kernel variable.
 - `argaddr()`: parses a pointer argument passed from the userspace to a syscall. This argument is usually a userspace pointer provided by the user, and you must use `copyin()` or `copyout()` to safely access that userspace address.
-

Task 2: System calls in xv6

A) Who is my parent?

Let's start with something straightforward. xv6 has a `getpid` syscall that returns the process ID of the calling process. Implement a new syscall `getppid` that returns the process ID of the parent of the calling process.

```
C/C++
int getppid(void);
```

Hints:

- Look at the implementation of `getpid` in `kernel/sysproc.c`. Implement your `sys_getppid` logic here.

- Assign a syscall number in `kernel/syscall.h`.
- Register the syscall in `kernel/syscall.c`.
- Add a userspace wrapper in `user/usys.pl`.
- Define the userspace function in `user/user.h`.

Sample output:

Use the provided `ppid.c` program to test your syscall:

```
Shell
$ ppid

[parent] my pid is 4
[parent] forking a child process
[child] my pid is 5
[child] my parent is 4
```

B) Square of a number

Add a syscall that takes an integer argument from the user, and computes its square.

```
C/C++
int square(int num);
```

Hints:

- Write the actual implementation for **square** syscall in `kernel/sysproc.c`. Look at existing syscall implementations in the same file for reference, and how they handle arguments provided by the user using `argint()`.
- Expose the syscall to userspace by updating `user/user.h` and `user/usys.pl`.
- *Assume the input to this syscall will be small enough to not overflow 32-bit integers for this lab.*

Sample output:

Use the provided `sq.c` program to test your syscall:

Shell

\$ sq

square of 2 is 4
square of 5 is 25

C) How many children?

xv6 does not keep a count of the number of children that a process has. Add a `child_count` field to the process control block (`struct proc`) to keep track of the number of (immediate) children a process currently has. Increment and decrement this counter based on the child process's lifecycle (forking and reaping of a child process). Next, add two syscalls to query this count from userspace:

C/C++

```
int get_child_count(void); // for current process
int get_process_child_count(int pid);
```

Hints:

- Find `struct proc` in `kernel/proc.h` and add your new field.
- Initialize the counter to zero when the process is allocated memory.
- Find relevant kernel functions in `kernel/proc.c` where this counter should be modified.
- When two or more child processes running on different CPUs are reaped at the same time, they will try to decrement the child counter of the parent process concurrently. Use **locking to prevent race conditions** when updating `struct proc`. Look at various functions in `kernel/proc.c` that involve locking the process structure.
- `get_process_child_count` should return `-1` if the process does not exist.
- The child counter should decrement when a child process is reaped, not when it exits.

Sample output:

Use the provided `test_children.c` program to test your implementation:

Shell

```
$ test_children
```

```
initial child count: 0
child_count after forking: 1
child_count after forking again: 2
child_count of pid 13: 1
child reaped. new child count: 1
child reaped. new child count: 0
```

D) Family Planning: Implement `nfork`

Implement a new syscall `nfork` such that the parent process can create multiple children using just one invocation.

C/C++

```
int nfork(int n, int* child_pids);
```

`nfork` should fork `n` children for the calling process and return

- `n` to the calling process
- `0` to each child process

The PID of children should be copied to the `child_pids` userspace address provided by the parent process. After forking, a child process should not execute the parent's logic in the kernel.

Hints:

- Use `agint`, `argaddr` to extract the arguments
- Use `kfork` to create child processes
- `copyout` can be used to copy child PIDs to userspace memory
- **Do not modify `kfork`**
- Ensure that the child counter from previous task is working as expected after implementing `nfork`

Sample output:

Use the provided `test_nfork.c` program to test your implementation:

```
Shell
$ test_nfork

[parent] pid: 3
[parent] forking 3 children
[child] pid: 5, ppid: 3
[child] pid: 6, ppid: 3
[child] pid: 7, ppid: 3
[parent] get_child_count returned 3
```

Use `getppid` from **Task A** to print the parent's PID. We'll also use `get_child_count` from **Task C** to verify the number of children forked.

E) Counting syscalls

Modify the kernel to maintain a counter for each kind of system call invoked by a process. Add a system call to print these counters (count of invocations for each syscall number).

```
C/C++
int print_syscalls(void); // for current process
int print_process_syscalls(int pid);
```

Hints:

- Add a fixed-size array to `struct proc` in `kernel/proc.h` to hold the counts. Size of this array is determined by the largest syscall number present in `kernel/syscall.h`.
- The function to augment with your syscall counter logic is present in `kernel/syscall.c`, identify it.

Sample output:

Use the provided `test_syscalls.c` program to test your implementation:

```
Shell
$ test_syscalls
Syscall counts for current process:
syscall_number      invocations
1          3
11         1

Syscall counts for pid 25
syscall_number      invocations
1          1
```

F) Working with files

When a process forks, the child inherits the parent's file descriptors. Do they share the same inode number and file read offsets? What happens to the parent's read offset and child reads from the same open file? Implement two new syscalls to fetch the inode number and the read offset of a file descriptor:

```
C/C++
uint64 get_inode_num(int fd);
uint64 get_read_offset(int fd);
```

Return -1 if the `fd` is invalid (or not of type `FD_INODE`) or the file is unreadable.

Write a user program that:

1. Opens a text file (with say file descriptor `F`)
2. Forks a child process. Now both the parent and child share `F`
3. Prints `F`'s inode numbers from both the parent and child process using `get_inode_num`
4. Reads some bytes from `F` in the child process
5. Prints the read offset in parent process using `get_read_offset`

Hints:

- Kernel maintains a list of open files as `ofile[]` field in `struct proc` for every process.
- Fields related to files are present in `struct file`

Sample output:

Use the provided `test_fd.c` program to test your implementation:

```
Shell
$ test_fd

[parent] fd=9, inode=2, offset=0
[parent] forking a child...
[child] fd=9, inode=2, offset=0
[child] reading 5 bytes from fd=9
[child] fd=9, inode=2, offset=5
[parent] reaping child...
[parent] fd=9, inode=2, offset=5
```

G) Peeking into files

Reading from an open file normally advances the read offset, meaning that the next read will pick up where the last one left off. Implement a **peek** syscall that allows for reading data from an open file without advancing the read offset. **peek** should copy the first `num_bytes` starting from the current read offset to the provided `user_addr` in userspace, without advancing the file's offset. Consecutive peek calls with the similar arguments should return the same values.

```
C/C++
int peek(int fd, char* user_addr, int num_bytes);
```

The syscall should copy `num_bytes` safely from kernel memory to the userspace memory pointed by `user_addr`. Return **-1** if the `fd` is invalid (or not of type `FD_INODE`) or the file is unreadable and **-2** if EOF has been reached.

Hints:

- Look at `sys_read` in `kernel/sysfile.c` and how it uses `fileread()` from `kernel/file.c`.
- You need a different kind of kernel helper function to parse pointer arguments. `argint()` cannot safely parse a pointer argument. Take a look at `kernel/sysproc.c` for pointers.

Sample output:

Use the provided `test_peek.c` program to test your implementation:

Shell

\$ test_peek

peeking 5 bytes: hello

peeking 5 bytes again: hello

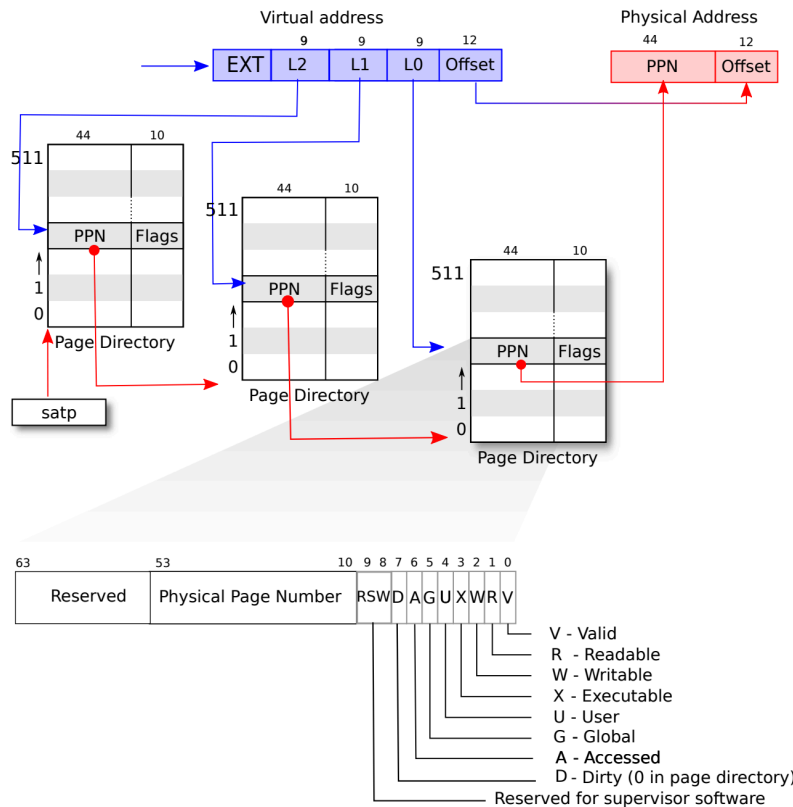
peeking 2 bytes: he

reading 2 bytes: he

peeking 3 bytes after reading: llo

Task 3: Virtual Memory & Page Tables

xv6 uses the RISC-V **Sv39** paging scheme to translate virtual addresses into physical addresses. As shown in **Figure**, a virtual address is divided into three 9-bit page-table indices (**L2**, **L1**, and **L0**) and a 12-bit page offset. The three indices are used to walk the three levels of the page-table hierarchy until the corresponding **Page Table Entry (PTE)** is reached. Each PTE contains a **physical page number (PPN)** along with **flags** that describe the mapping.



The **PTE_V (valid)** flag indicates whether a PTE represents a valid mapping. If PTE_V is not set, the mapping is invalid and the corresponding virtual address cannot be translated through that PTE. The other important PTE flags specify the permitted type of access: **PTE_R** allows reading, **PTE_W** allows writing, **PTE_X** allows instruction execution, and **PTE_U** determines whether the page can be accessed from user mode. Thus, the PTE provides both the **destination** of the mapping (PPN) and the **permissions** associated with it.

For a **virtual-to-physical address translation**, the hardware starts from the root page table identified

by **satp**. The L2, L1, and L0 portions of the virtual address are successively used to select entries at the three levels of the page-table tree. The final PTE supplies the physical page number.

The original 12-bit page offset is then combined with this PPN to form the physical address. xv6's `walk()` function follows the same three-level procedure as the RISC-V hardware and returns the address of the relevant PTE.

Functions to Refer: Students may refer to `ismapped()`, `walk()` and `walkaddr()` in `kernel/vm.c`, and `PX()`, `PTE2PA()`, `PTE_FLAGS()`, and the PTE flag definitions (**PTE_V**, **PTE_R**, **PTE_W**, **PTE_X**, **PTE_U**) in `kernel/riscv.h` to understand PTE validity, permissions, and virtual-to-physical address translation.

A. VA-lidation: PTE Validity

For this task, implement a system call that determines whether a specific virtual address is valid and backed by physical memory.

```
int pte_valid(uint64 va);
```

This function should take a virtual address (va) as an argument. It must return 1 if the address maps to a valid physical page and 0 if it does not.

We have provided a sample program, `pte.c`, which uses this system call. You can use this to verify your implementation.

Shell

```
$ pte
```

```
VA: 0x00000000000003FBC valid: 1
```

```
VA: 0x0000000100000000 valid: 0
```

Refer to: `kernel/vm.c` (`walk()`, `ismapped()`), `kernel/syscall.c` and existing system call implementations.

Files to Modify: `user/user.h`, `user/usys.pl`, `kernel/sysproc.c`, `kernel/syscall.h`, `kernel/syscall.c`, `kernel/defs.h`, `kernel/vm.c`.

B. Flag & Furious: PTE flags

Just as a virtual address can be mapped (valid) or unmapped (invalid), there are other permission information items (bits in the pte flag) that govern memory access.

```
void get_pteflags(uint64 va);
```

This function takes a virtual address (va) as an argument. It should look up the corresponding page table entry and print whether the address is:

- Readable (R)
- Writable (W)
- Executable (X)
- User Accessible (U)

Hints: Refer to the `walk()` logic in `kernel/vm.c` to understand how to access page-table entries, and inspect `kernel/riscv.h` for PTE flags such as `PTE_V`, `PTE_R`, `PTE_W`, and `PTE_X` to identify valid leaf mappings. Implement the PTE lookup logic in `kernel/proc.c`. Use the **current process's page table** and `walk()` to locate the PTE corresponding to the given virtual address

Sample output:

Use the provided `pflag.c` program to test your implementation:

```
Shell
$ pflag
VA: 0x0000000000000000 -> R:1 W:0 X:1 U:1
VA: 0x00000000000003fac -> R:1 W:1 X:0 U:1
VA: 0x00000000000004000 -> R:1 W:1 X:0 U:1
VA: 0x00000000000001000 -> R:1 W:1 X:0 U:1
VA: 0x00000000000000900 -> R:1 W:0 X:1 U:1
VA: 0x00000003ffffff000 -> R:1 W:0 X:1 U:0
```

Review the provided program to understand which virtual addresses are being checked and the corresponding **PTE** flags for each address.

C. Uncovering the Physical Address: `va2pa`

Write a system call `va2pa()` that returns the virtual address to physical address mapping from the page table of the current process. Specifically, the system call should have the following interface:

```
uint va2pa(uint virtual_addr);
```

This function should take **virtual address** as an argument to the call and corresponding **physical address** is the return value.

Hint: Refer to the page-table traversal and mapping logic in `kernel/vm.c`, particularly `walk()`, `walkaddr()`, and `PTE_V`. Also inspect `struct proc` in `kernel/proc.h` to determine how to access the current process's page table.

Use the provided `t1_va2pa.c`, `t2_va2pa.c` and `t3_va2pa.c` program to test your implementation:

Sample output:

```
Shell

$ t1_va2pa
VA1: 0x4000 -> PA1: 0x87F26000
VA2: 0x5000 -> PA2: 0x87F54000

$ t2_va2pa
Virtual address of variable N in child : 0x3FBC
Physical address of variable N in child : 0x87F2EFBC
```

```
Virtual address of variable N in parent : 0x3FBC
Physical address of variable N in parent : 0x87F47FBC
$ t3_va2pa
VA1: 0x4000 -> PA1: 0x87F43000
VA2: 0x4064 -> PA2: 0x87F43064
```

Run the three provided test programs and carefully examine their outputs. Based on the virtual and physical addresses reported by each program:

- **Program 1:** Why do **VA1** and **VA2** map to different physical addresses?
- **Program 2:** Why does the **same virtual address** of variable **N** map to different physical addresses in the parent and child?
- **Program 3:** Why do **VA1** and **VA2** map to physical addresses with the same page base but different offsets?

From these three cases, identify **which parts of a virtual address are changed and which parts are preserved** during VA → PA translation.

D. Virtual Address Space Explorer

Write a system call **getvasize()** that returns the size of the address space used by a process. Specifically, the system call should have the following interface:

```
int getvasize(int pid);
```

This function should take **pid** of a process as an argument to the call and amount of virtual memory used by the process is the return value.

We have provided a sample program, **v_getvasize.c**, which uses this system call. You can use this to verify your implementation.

```
Shell
$ v_getvasize
Pid of the process is 3
Size of process:      16384 Bytes
Address returned by sbrk: 0x0000000000004000
Size of process:      17408 Bytes
Address returned by sbrk: 0x0000000000004000
```

Look up and understand implementation of the **sbrk** system call in **proc.c**. Also check the struct **proc** data structure in **proc.h**. The **sbrk(n)** system call is implemented in the function **sys_sbrk()** in **sysproc.c** that allocates physical memory and maps it into the process's virtual address space. The **sbrk(n)** system call grows the process's memory size by **n** bytes, and then returns the start of the newly allocated region (i.e., the old size).

Submission Guidelines (TODO)

- **Deadline: 24th August 2026, 5 pm. (via Moodle)**
-

Appendix A

Follow these steps for installing xv6 on a Ubuntu-based Linux machine:

1. Prerequisites
 - a. First, update your system:
sudo apt update
 - b. Install the required packages:
sudo apt install -y git build-essential qemu-system-riscv64 gcc-riscv64-unknown-elf
 - c. **build-essential**
For essential packages like libc, gcc, make, etc.
 - d. **qemu-system-riscv64**
Emulates a RISC-V machine in software.
Your computer's real CPU speaks x86-64, but xv6 is written for RISC-V.
QEMU **pretends to be a RISC-V computer**, so the OS can run even though the hardware is different.
 - e. **gcc-riscv64-unknown-elf**
It runs on your x86-64 machine but produces **RISC-V binaries**, not x86-64 ones.
We need this because xv6 must be compiled for the **CPU architecture that QEMU is emulating**.
2. Verify toolchain installation
riscv64-unknown-elf-gcc --version
qemu-system-riscv64 --version
If both commands work, your environment is ready.

3. Clone the xv6 RISC-V Repository
git clone <https://github.com/mit-pdos/xv6-riscv.git>
cd xv6-riscv
4. Build xv6
make
Run the command to build the OS. If compilation succeeds, you are ready to run xv6.
5. Run xv6 on QEMU
Start the xv6 operating system:
make qemu

Appendix B

A list of risc-v ISA items and xv6 items that intersect with the xv6 system call implementation —
For details of these and more look up Chapter 4 of the xv6 book.

None

`ecall`

`sret`

`stvec`

`sepc`

`scause`

`sstatus` (SIE and SPP)

`sscratch`

`satp`

`csrw`

`a0`

`TRAMPOLINE`

`TRAPFRAME`

`uservec`

`usertrap`

```
userret  
prepare_return  
syscall  
copyin  
copyout  
copyinstr  
argint  
argstr
```